Sprint 2 Plan

SlugFound

CSE 115

Sprint Completion Date: TBD
Revision: 1.0
Revision Date: April 27, 2025

1. Sprint Goal

Integrate the backend foundation into the SlugFound front-end prototype by implementing user authentication via Supabase (email/password), designing and deploying the database schema for lost and found item posts, and wiring the post creation form to persist real data. By the end of this sprint, users should be able to register, sign in, create posts that are saved to the database, and view real posts on the Found and Lost Items pages.

2. Task Listing (by User Story)

Total estimated hours this sprint: 40 hours

User Story 2.1 [High] [5 pts]

"As a new UCSC student, I want to create an account using my email and password so that I can post and track lost and found items tied to my identity."

Specifications & Requirements

- **Auth provider:** Supabase Auth with email/password sign-up and email confirmation.
- **Email validation:** Enforce @ucsc.edu email domain on registration. Display inline error if non-UCSC email is entered.
- **Password requirements:** Minimum 8 characters, at least one uppercase letter and one number. Show real-time strength indicator.
- **User profile row:** On successful registration, create a row in a public.profiles table (id, display_name, email, avatar_url, created_at) linked to auth.users via foreign key.
- **UI:** Sign-up page with email field, password field with strength indicator, confirm password field, display name field, and a link to the sign-in page. Match existing landing page design system.
- **Error handling:** Handle duplicate email, weak password, confirmation mismatch, and network errors with user-facing toast notifications.

Tasks

- Configure Supabase project and enable email/password auth provider in dashboard *(1 hrs)*
- Create profiles table in Supabase with RLS policies (users can read all, update own) *(2 hrs)*
- Build sign-up page UI with email, password, confirm password, display name, and validation *(3 hrs)*

- Implement Supabase signUp() call with email confirmation flow and profile row creation *(2 hrs)*
- Add error handling with toast notifications for all failure cases *(1 hrs)*

Total for User Story 2.1: 9 hours

Definition of Done

- ☐ User can register with a @ucsc.edu email and receive a confirmation email
- ☐ A profiles row is created for every new user with display_name and email populated
- ☐ Non-UCSC emails are rejected with a clear inline error message
- ☐ Passwords that don't meet requirements show real-time validation feedback
- ☐ Duplicate email registration shows a descriptive error, not a generic failure
- ☐ All sign-up code is merged to main branch on GitHub with a passing build

User Story 2.2 [High] [5 pts]

"As a returning user, I want to sign in with my email and password so that I can access my account and see my previous posts."

Specifications & Requirements

- **Sign-in method:** Email/password via Supabase signInWithPassword(). Redirect to the Found Items page on success.
- **Session management:** Use Supabase client-side session with automatic token refresh. Store session in a React context provider (AuthContext) accessible app-wide.
- **Protected routes:** Wrap all app pages (Found, Lost, Post, Messages, Account) in an auth guard. Redirect unauthenticated users to the landing page.
- **Sign-out:** Account page sign-out button calls Supabase signOut(), clears session context, and redirects to landing page.
- **Persistent session:** On page reload, check for existing Supabase session and auto-restore. Show a loading spinner while session is being verified.
- **UI updates:** Display signed-in user's display_name and initials avatar in the sidebar. Update Account page to show real profile data instead of hardcoded mock.
- **Error handling:** Handle incorrect password, unconfirmed email, account not found, and network errors with specific user-facing messages.

Tasks

- Build sign-in page UI with email/password form and link to sign-up page *(2 hrs)*
- Implement signInWithPassword() with redirect logic and error handling *(2 hrs)*
- Create AuthContext provider with session state, loading state, and auto-refresh *(3 hrs)*
- Implement protected route wrapper that redirects unauthenticated users *(2 hrs)*
- Wire sign-out button to Supabase signOut() and clear session context *(1 hrs)*
- Update sidebar and Account page to display real user data from AuthContext *(1 hrs)*

Total for User Story 2.2: 11 hours

Definition of Done

- ☐ User can sign in with email/password and is redirected to Found Items page
- ☐ Incorrect password or unconfirmed email shows a specific, helpful error message

- ☐ Session persists across page reloads without requiring re-authentication
- ☐ Unauthenticated users attempting to access /found, /lost, /post, /messages, or /account are redirected to landing
- ☐ Sign-out clears session and returns user to landing page
- ☐ Sidebar shows the signed-in user's name and initials (not hardcoded "Bardia N.")
- ☐ All auth code is merged to main on GitHub with a passing build

User Story 2.3 [High] [5 pts]

"As a developer, I want a well-structured database schema so that lost and found item posts are stored reliably with proper relationships to users."

Specifications & Requirements

- **Items table:** public.items table with columns: id (uuid, PK), user_id (FK to profiles.id), type (enum: 'lost' | 'found'), title (text, max 120 chars), description (text, max 1000 chars), category (text, one of: Electronics, Clothing, Accessories, Books, Keys, ID/Cards, Water Bottle, Other), location (text, one of predefined UCSC campus locations), status (enum: 'active' | 'claimed' | 'resolved'), image_url (text, nullable), created_at (timestamptz), updated_at (timestamptz).
- **Row-level security:** RLS enabled on items table. Policies: anyone authenticated can SELECT all items; only the item owner can INSERT, UPDATE, and DELETE their own items.
- **Indexes:** Create indexes on type, category, status, and created_at for efficient filtering and sorting.
- **Seed data:** Create a SQL seed script with 10+ sample items (mix of lost and found, various categories and locations) for development and demo purposes.
- **Migration:** All schema changes managed via Supabase SQL migrations committed to the GitHub repo under /supabase/migrations/.

Tasks

- Design items table schema and write CREATE TABLE migration SQL *(2 hrs)*
- Write RLS policies for items table (select all, insert/update/delete own) *(2 hrs)*
- Create indexes on type, category, status, created_at columns *(1 hrs)*
- Write seed data SQL script with 10+ sample items across categories *(1 hrs)*
- Test schema by running migrations on a fresh Supabase project and verifying RLS *(1 hrs)*

Total for User Story 2.3: 7 hours

Definition of Done

- ☐ items table exists in Supabase with all specified columns, types, and constraints
- ☐ RLS policies prevent users from modifying other users' items (verified via Supabase SQL editor)
- ☐ Seed script runs without errors and populates 10+ items visible in the Supabase dashboard
- ☐ All migration files are committed to /supabase/migrations/ in the GitHub repo
- ☐ A developer can clone the repo, run migrations, and have a working database with seed data

User Story 2.4 [High] [5 pts]

"As a logged-in user, I want to submit a lost or found item post so that it is saved to the database and appears on the appropriate listings page."

Specifications & Requirements

- **Form submission:** POST the form data to Supabase items table via supabase.from('items').insert(). Attach the authenticated user's ID automatically from the session context.
- **Validation:** Client-side: require title, category, location, and description. Server-side: Supabase RLS ensures user_id matches authenticated user.
- **Image upload:** Upload image to Supabase Storage bucket 'item-images'. Generate a public URL and store it in the image_url column. Max file size 5MB, accepted formats: JPG, PNG, WebP.
- **Success flow:** On successful submission, show a success toast, clear the form, and redirect to the Found or Lost Items page (depending on post type).
- **Loading state:** Disable submit button and show a spinner during submission to prevent duplicate posts.
- **Error handling:** Handle image upload failures separately from database insert failures. Show specific error messages for each.

Tasks

- Wire post form submit to Supabase items.insert() with user_id from AuthContext *(2 hrs)*
- Set up Supabase Storage bucket 'item-images' with public access and 5MB size limit *(1 hrs)*
- Implement image upload to Storage with progress indicator and public URL generation *(3 hrs)*
- Add loading state, success toast, form reset, and redirect on successful submission *(1 hrs)*
- Add error handling for validation failures, upload failures, and insert failures *(1 hrs)*

Total for User Story 2.4: 8 hours

Definition of Done

- ☐ Submitting the form creates a row in the items table with all fields correctly populated
- ☐ The user_id column is automatically set to the authenticated user (not manually entered)
- ☐ Uploading an image stores it in Supabase Storage and saves the URL to the item row
- ☐ Submitting without required fields shows inline validation errors (no database call made)
- ☐ After successful submission, the user is redirected and can see their post on the listings page
- ☐ Submitting a post without an image works (image_url is null)
- ☐ All code merged to main on GitHub with a passing build

User Story 2.5 [Medium] [3 pts]

"As a user, I want the Found Items and Lost Items pages to show real posts from the database so that I can browse actual items students have posted."

Specifications & Requirements

- **Data fetching:** Replace hardcoded mock data with Supabase queries: supabase.from('items').select().eq('type', 'found') for Found page, .eq('type', 'lost') for Lost page. Order by created_at descending (newest first).
- **Search:** Implement search using Supabase .ilike() on title and description columns. Debounce search input by 300ms.

- **Category filter:** Filter by category using .eq('category', selectedCategory). Reset to all items when "All" is selected.
- **Loading state:** Show a skeleton loading state while items are being fetched. Show an empty state message if no items exist.
- **Item card updates:** Display the poster's display_name (join with profiles table). Show uploaded image if image_url exists, otherwise keep the photo placeholder.
- **Real-time (stretch):** If time allows, subscribe to Supabase real-time changes on the items table so new posts appear without page refresh.

Tasks

- Replace mock data with Supabase select queries on Found and Lost pages *(2 hrs)*
- Implement debounced search with .ilike() and category filter with .eq() *(2 hrs)*
- Add skeleton loading states and empty state messaging *(1 hrs)*
- Update item card to display real image and poster name (join profiles) *(2 hrs)*

Total for User Story 2.5: 7 hours

Definition of Done

- ☐ Found Items page displays only items with type='found' from the database, newest first
- ☐ Lost Items page displays only items with type='lost' from the database, newest first
- ☐ Searching filters results in real-time (debounced) across title and description
- ☐ Category dropdown filters results correctly and resets when "All" is selected
- ☐ A newly created post appears on the appropriate page without manual page refresh
- ☐ Item cards show the real poster's name and uploaded image (or placeholder if none)
- ☐ All code merged to main on GitHub with a passing build

User Story 2.6 [Low] [2 pts]

"As a logged-in user, I want my Account page to show my real profile information and post statistics so that I can see my activity on the platform."

Specifications & Requirements

- **Profile data:** Fetch display_name, email, avatar_url, and created_at from the profiles table. Display user's initials as avatar.
- **Post stats:** Query items table to count: total posts by user, items with status='claimed' or 'resolved'. Display as stat cards.
- **Edit profile (stretch):** If time allows, allow user to update display_name via an inline edit field that calls supabase.from('profiles').update().

Tasks

- Fetch and display real profile data from profiles table on Account page *(1 hrs)*
- Query and display post count and reunited count stats from items table *(2 hrs)*
- Generate initials avatar from display_name and style to match design system *(1 hrs)*

Total for User Story 2.6: 4 hours

Definition of Done

- ☐ Account page shows the signed-in user's real name, email, and join date
- ☐ Post count and reunited count reflect actual data from the items table

- ☐ User sees an initials-based avatar generated from their display name
- ☐ All code merged to main on GitHub with a passing build

3. Team Roles

Team Member	Email	Role(s)
Bardia Nasab	bnasab@ucsc.edu	Scrum Master
Colin Posat	cposat@ucsc.edu	Developer, UI/UX Lead
Sam Madinya	smadinya@ucsc.edu	Developer
Aidan Nguyen	anguy480@ucsc.edu	Developer

4. Initial Task Assignment

Team Member	User Story	Initial Task
Bardia Nasab	US 2.1	Configure Supabase project and enable auth providers
Colin Posat	US 2.2	Create AuthContext provider with session state and auto-refresh
Sam Madinya	US 2.3	Design items table schema and write CREATE TABLE migration
Aidan Nguyen	US 2.4	Wire post form submit to Supabase items.insert()

5. Initial Scrum Board

The Sprint 2 scrum board is managed in Jira.

6. Initial Burnup Chart

The Sprint 2 burnup chart is tracked in Jira and will be updated as tasks are completed.
Total sprint capacity: 25 story points across 40 estimated hours.

7. Scrum Meeting Times

Meeting #	Day	Time	Location	TA/Tutor Visit?
1	Mondays	12:00 – 1:00 PM	Zoom	Yes — Lab session
2	Wednesdays	5:30 – 6:30 PM	TBD	No
3	Fridays	12:00 – 1:00 PM	TBD	No